

1^{er} Parcial - Técnica y Diseño de Algoritmos (ex Algo3)
29 de Sept. 2025 - 2^{do} Cuatrimestre

Nombre y Apellido	#Orden	L.U.	Turno	Corrector	Nota
				Dafne	

Duración: 4 horas. Este examen es individual y a libro cerrado. La nota de aprobación es de 60/100 puntos

Solo Ej. 3 y 4 pueden responderse en hoja aparte

Ej 1: (12 Puntos) Marque la/s opción/es correcta/s. En el punto (3), marque la opción (C) si cree valen tanto (A) como (B). Puntajes: Correcto: 2 pts | Parcial: 0 pts | Incorrecto: -1 pt 9/12

I. En el paradigma divide y vencerás, ¿qué condiciones deben cumplir para que el costo de división+combinación domine la complejidad total del algoritmo?	IV. ¿Qué determina principalmente la efectividad de la poda en backtracking?
A) Cuando el costo de combinación es mayor o igual que el de división	A) El número total de variables
B) Cuando el trabajo de división+combinación crece más rápido que el trabajo total de los subproblemas	B) Qué tan temprano se pueden detectar las violaciones de restricciones
C) Cuando hay más de dos subproblemas	C) La profundidad del árbol de búsqueda
D) Cuando los subproblemas no son de igual tamaño	D) El orden de las soluciones en el espacio
II. Marque el orden de complejidad correcto de la recurrencia $T(n) = 16T(n/4) + n$	V. La propiedad de elección greedy significa:
A) $\Theta(n \log n)$	A) Cualquier elección lleva a una solución óptima
B) $\Theta(\log^2 n)$	B) La elección localmente óptima lleva a una solución globalmente óptima
C) $\Theta(n^2 \log n)$	C) Las elecciones pueden deshacerse si es necesario
D) $\Theta(n^2)$	D) Existen múltiples soluciones óptimas
III. ¿Cómo se aplica programación dinámica si el problema en cuestión es:	VI. Marcar los sistemas de monedas donde el greedy funcionaría
A) Se pueden encontrar soluciones óptimas de subproblemas para encontrar la solución óptima del problema	A) Las monedas son de cualquier denominación
B) Se usa un mismo subproblema para resolver diferentes problemas mayores	B) Monedas de 1, 3 y 5 centavos
C) Los dos anteriores	C) Monedas de 2, 4 y 6 centavos
D) Ninguna propiedad	D) Monedas de 1, 2, 7 y 10 centavos

Ej 2: (18 Puntos) Para cada uno de los siguientes problemas, determine la técnica (Divide & Conquer, Greedy, Dinámica Top-Down, Dinámica Bottom-Up) que emplearía para resolverlo teniendo en cuenta los siguientes factores ordenados de mayor a menor importancia: correctitud, complejidad/eficiencia temporal, complejidad/eficiencia espacial y simplicidad. Justifique su elección y/o la no elección de las otras técnicas de acuerdo a estos factores. Cada item aporta 6 pts. a la nota.

I) Fibonacci

Dinámica Bottom-Up: por qué? pues como para Fib solo necesito los dos valores inmediatamente anteriores, este enfoque me permite construir desde mis casos bases. Siendo bueno en espacio y tiempo. ¿qué complejidad? otras técnicas?

II) Problema del cambio (dar un vuelto con menor cantidad de monedas, hay monedas de k diferentes valores con disponibilidad infinita)

Dinámica Top-Down: puedo ir seleccionando cada m moneda de mis K valores, resolver para ese sub-problema (vuelto - m) y elegir al menor cantidad me de. Va a haber superposición de estados por lo que dinámica es bueno.

III) Multiplicación de 2 números enteros de n bits

Greedy: vamos multiplicando al número mas significativo y después sumamos respetando sus posiciones.

Ej 3: (20 Puntos) Dada una lista n de tareas con un determinado orden, donde cada tarea $i = 1, \dots, n$ toma t_i unidades de tiempo en ser completada. Sea $demora_i = \sum_{j=1}^i t_j$ el tiempo que se debe esperar para que la i-ésima tarea sea completada (es decir, para completar cada tarea es necesario que se completen todas las anteriores cada vez). Nos interesa elegir un orden en el cual resolver las tareas de manera que se minimice la sumatoria de demoras de todas:

- Plantear cómo sería el orden de las tareas en un algoritmo greedy (5pts)
- Demuestre que ese algoritmo greedy es correcto. Su demostración debe mostrar que no existe ninguna solución que requiera menos demora total que la obtenida por el algoritmo greedy (15pts)

Ej 4: (30 Puntos) Queremos contar cuántas combinaciones distintas de secuencias $v = (v_1, v_2, \dots, v_n)$ existen tal que $v_i \in \mathbb{N}$, $1 \leq v_i \leq k$ y $\sum_{i=1}^n v_i = s$, donde n , k y s son parámetros. El algoritmo diseñado debe estar basado en la técnica de Programación Dinámica. Considere a k como constante. Note que se debe cumplir $n \leq s \leq n \cdot k$ para que la respuesta no sea trivial.

- Definir de forma recursiva la función $\text{combinaciones}(n, k, s)$ que calcule la cantidad de formas de conseguir la suma s con las restricciones explicadas. No olvide definir los casos base (10 pts).
- Demostrar que se cumple la propiedad de superposición de subproblemas (10 pts).
- Definir un algoritmo (escribir pseudocódigo) *top-down* para calcular $\text{combinaciones}(n, k, s)$ indicando claramente las estructuras de datos utilizadas y la complejidad temporal y espacial resultantes (10 pts).

Ej 5: (10 Puntos) Analice la complejidad temporal del siguiente algoritmo usando el Teorema Maestro:

```
def es_derecha_dominante(arr, inicio, fin):
    if fin - inicio == 1:
        return True, arr[inicio]

    medio = (inicio + fin) // 2

    izq_dominante, suma_izq = es_derecha_dominante(arr, inicio, medio)
    der_dominante, suma_der = es_derecha_dominante(arr, medio, fin)

    es_dominante = (suma_der > suma_izq) and izq_dominante and der_dominante
    suma_total = suma_izq + suma_der

    return es_dominante, suma_total
```

Describa a (número de subproblemas), b (factor de división), $f(n)$ (costo de dividir+combinar), caso del Teorema Maestro y complejidad final.

$a = 2$ por procesar 2 mitades.
 $b = 2$ por reduce a la mitad el array.
 $f(n) = O(1)$ por solo hacer operaciones booleanas y aritmeticas elementales.

Caso 1:

$$f(n) = O(1) \in O(n^{\log_2 2}) = O(n^1)$$

$$T(n) = O(n^{\log_2 2}) = O(n)$$

(B)

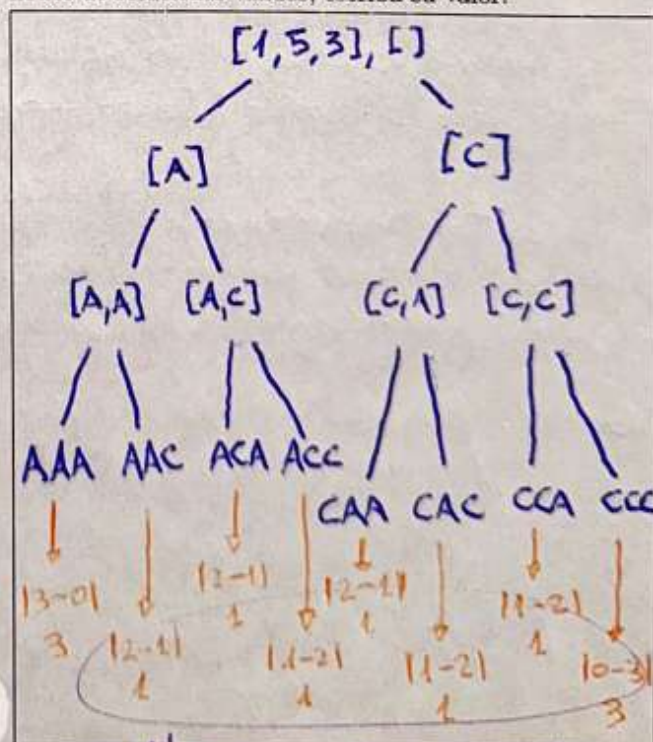
Ej 6: (10 Puntos) El parcial terminó y tenés que repartir las n entregas entre dos ayudantes (Alicia y Carlos) para que corrijan. Cada parcial tiene una cantidad de hojas h . Tu objetivo es minimizar la diferencia entre la cantidad de hojas que debe corregir cada ayudante. Para ver cual es la mínima diferencia de hojas que podés conseguir, hacés el siguiente algoritmo de backtracking:

```
1 def particion(h, solucion):
2     if len(solucion) == len(h):
3         return diferencia(h, solucion)
4     part_a = solucion + ['A']
5     part_c = solucion + ['C']
6     return min(particion(h, part_a),
7                particion(h, part_c))
8
9 def diferencia(h, solucion):
10    sum_a = 0
11    sum_c = 0
12    for i in range(len(h)):
13        if solucion[i] == 'A':
14            sum_a += h[i]
15        if solucion[i] == 'C':
16            sum_c += h[i]
17    return abs(sum_a - sum_c)
```

Asumiendo que las líneas 4 y 5 son $O(1)$, reporte la complejidad del algoritmo.

$$O(2^{14} \cdot 14!)$$

Dibuje el árbol de recursión con $h = [1, 5, 3]$. Para los nodos de soluciones válidas, escriba su valor.



No son estos los valores
 SE ENTIENDE LA IDEA

(A) (B)

ES 3

a)

Algo que tenemos que minimizar:

$$\sum_{i=1}^n \left(\sum_{j=1}^i t_j \right)$$

Mos 3/3

Me conviene que los primeros terminos de $\sum_{i=1}^n$ sean los mas chicos posibles, pues ellos son los que mas veces van a ser sumados

y para que los terminos sean pequeños necesitamos que $\sum_{j=1}^i$ sea pequeño para los primeros "i"s.

Entonces el orden de tareas para greedy debería ser t ascendente. Es decir, ordenados de forma ascendente segun el tiempo que les toma completarse.

(B)

b)

Problemas usando "greedy stays ahead" que es una estrategia donde partiendo de una solución O óptima cualquiera, nuestra algoritmo greedy brinda una solución al menos tan buena como O.

$$G_i = \{g_1, \dots, g_n\}$$
$$O_i = \{o_1, \dots, o_n\}$$

(M)

no llegamos

ES 4

A partir de ejemplos:

Mosa 1/3

con $K=4 \rightarrow 1 \leq N_i \leq 4$
con $S=5$
con $n=2$



$4 + 1$
 $3 + 2$
 $2 + 3$
 $1 + 4$

4 comb.

Siendo $K=4$ constante, ahora con $n=3$:

$3+2+1=5$ comb

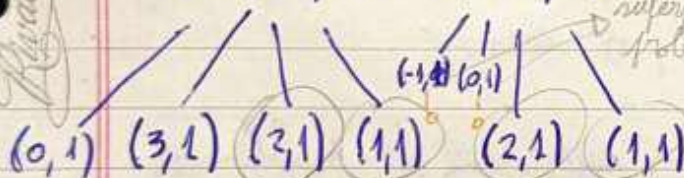
comb(5, 3)

comb(4, 2)

comb(3, 2)

comb(2, 2)

comb(1, 2)



$(2=1+1)$
 $(2=1+1)$

0 comb.

1 combinación posible
(3=3)

com base

↳ el árbol es del orden de tener n niveles y en cada nivel, cada nodo, se abre en K opciones.
es decir, cont. nodos $O(K^n)$

Además, en este caso ejemplo, ya se empieza a ver la superposición de problemas en los sub-problemas comb(2, 1) y comb(1, 1).

a)

$$\text{comb}(s, k, n) = \begin{cases} 0 & \text{si } s < n \\ 1 & \text{si } s = n \\ 1 & \text{si } n = 1 \\ \sum_{i=1}^k \text{comb}(s-i, k, n-1) & \text{si no} \end{cases}$$

Es decir, cualquier secuencia suma s , porque estás diciendo que si tiene 1 elemento alcanza para saber que suma s .

(B)

b) ¿Por qué?

Se puede observar que la cont. de llamadas que se va a hacer a la función es del orden $O(k^n)$. $O(n \cdot s)$ porque k cte.

Sin embargo, la función tiene $O(n \cdot k)$ estados (si bien la función tiene 3 argumentos como 1 de ellos es constante, no nos interesa a la hora de hacer memoization)

Son $O(n \cdot k)$ estados porque k restos, n veces.

Como $n \cdot k \ll k^n$, entonces hay superposición de problemas.

(B)

c)

La estructura de memoization es un array de 2 dimensiones. ✓

memo[][] ← inicializo con -1 ✓

comb(s, k, n):

c-total ← 0

si $s == n$ || $n == 1$:

devolver 1

si $s < n$:

devolver 0

for ($i = 1, i \leq k, i++$):

si memo[s-i][n-1] == -1:

memo[s-i][n-1] ← comb(s-i, k, n-1)

c-total ← c-total + memo[s-i][n-1]

devolver c-total.

(B)

Continuación E54-C:

Complejidad Temporal: es del $O(n \cdot n)$ ^{No} ya que el ~~es~~ trabajo que hacemos en cada estado es el for ~~hacer~~ n , además de el ~~inici~~ inicializar memo.

Complejidad espacial: es del $O(n \cdot n)$ el tamaño de la matriz memo.

(R)

Mosa 2/3